

ÉCOLE D'INGÉNIEURS SUP GALILÉE

Rapport du TP2 de Base De Données Avancées

Encadrant :
Mr YUCEF

Élève :
Maryse GOEH-AKUE
N° étudiant : 12101028

1 Exercice 1 : Requêtes SQL

Dans cette partie, chaque question a été exécutée dans Oracle APEX. Pour chaque question, la première capture correspond à la requête SQL et la seconde au résultat obtenu.

Q1. Afficher le nom du département qui a le budget le plus élevé

Cette requête permet d'identifier le département dont le budget est maximal.

```
SELECT dept_name
FROM department
WHERE budget = (
    SELECT MAX(budget)
    FROM department
);
```

FIGURE 1 – Script SQL de la question 1

DEPT_NAME
Finance

FIGURE 2 – Résultat de la question 1

Q2. Afficher les salaires et les noms des enseignants qui gagnent plus que le salaire moyen

Cette requête compare chaque salaire à la moyenne des salaires des enseignants.

```
SELECT name, salary
FROM teacher
WHERE salary > (
    SELECT AVG(salary)
    FROM teacher
);
```

FIGURE 3 – Script SQL de la question 2

NAME	SALARY
Wu	90000
Einstein	95000
Gold	87000
Katz	75000
Singh	80000
Brandt	92000
Kim	80000

FIGURE 4 – Résultat de la question 2

Q3. Pour chaque enseignant, afficher les étudiants ayant suivi au moins deux cours dispensés par cet enseignant, avec HAVING

Cette requête regroupe les couples enseignant-étudiant et conserve ceux pour lesquels le nombre de cours communs est supérieur ou égal à deux.

```

SELECT t.name AS teacher_name,
       s.name AS student_name,
       COUNT(*) AS total_courses
FROM teacher t
JOIN teaches te
  ON t.id = te.id
JOIN takes ta
  ON te.course_id = ta.course_id
  AND te.sec_id = ta.sec_id
  AND te.semester = ta.semester
  AND te.year = ta.year
JOIN student s
  ON s.id = ta.id
GROUP BY t.name, s.name
HAVING COUNT(*) >= 2
ORDER BY t.name, s.name;

```

FIGURE 5 – Script SQL de la question 3

Results Explain Describe Saved SQL History		
TEACHER_NAME	STUDENT_NAME	TOTAL_COURSES
Crick	Tanaka	2
Katz	Levy	2
Srinivasan	Bourikas	2
Srinivasan	Shankar	3
Srinivasan	Zhang	2

5 rows returned in 0.13 seconds [Download](#)

FIGURE 6 – Résultat de la question 3

Q4. Même question sans utiliser HAVING

Ici, le filtrage sur le nombre de cours est réalisé à l'extérieur de la sous-requête.

```
SELECT teacher_name, student_name, total_courses
FROM (
    SELECT t.name AS teacher_name,
           s.name AS student_name,
           COUNT(*) AS total_courses
    FROM teacher t
    JOIN teaches te
      ON t.id = te.id
    JOIN takes ta
      ON te.course_id = ta.course_id
      AND te.sec_id = ta.sec_id
      AND te.semester = ta.semester
      AND te.year = ta.year
    JOIN student s
      ON s.id = ta.id
    GROUP BY t.name, s.name
)
WHERE total_courses >= 2
ORDER BY teacher_name, student_name;
```

FIGURE 7 – Script SQL de la question 4

TEACHER_NAME	STUDENT_NAME	TOTAL_COURSES
Crick	Tanaka	2
Katz	Levy	2
Srinivasan	Bourikas	2
Srinivasan	Shankar	3
Srinivasan	Zhang	2

FIGURE 8 – Résultat de la question 4

Q5. Afficher les identifiants et les noms des étudiants qui n'ont pas suivi de cours avant 2010

Cette requête élimine les étudiants ayant suivi au moins un cours avant l'année 2010.

```
SELECT id, name
FROM student
MINUS
SELECT s.id, s.name
FROM student s
JOIN takes t
  ON s.id = t.id
WHERE t.year < 2010;
```

FIGURE 9 – Script SQL de la question 5

ID	NAME
23121	Chavez
70557	Snow
55739	Sanchez
19991	Brandt

4 rows returned in 0.02 seconds [Download](#)

FIGURE 10 – Résultat de la question 5

Q6. Afficher tous les enseignants dont les noms commencent par E

La condition utilisée permet de sélectionner les noms commençant par la lettre E.

```
SELECT *
FROM teacher
WHERE name LIKE 'E%';
```

FIGURE 11 – Script SQL de la question 6

ID	NAME	DEPT_NAME	SALARY
22222	Einstein	Physics	95000
32343	El Said	History	60000

FIGURE 12 – Résultat de la question 6

Q7. Afficher les salaires et les noms des enseignants qui perçoivent le quatrième salaire le plus élevé

Cette requête compare les salaires distincts pour retrouver le quatrième salaire le plus élevé.

```
SELECT name, salary
FROM teacher t1
WHERE 3 = (
    SELECT COUNT(DISTINCT t2.salary)
    FROM teacher t2
    WHERE t2.salary > t1.salary
);
```

FIGURE 13 – Script SQL de la question 7

NAME	SALARY
Gold	87000

FIGURE 14 – Résultat de la question 7

Q8. Afficher les noms et les salaires des trois enseignants qui perçoivent les salaires les moins élevés, par ordre décroissant

On récupère ici les trois plus petits salaires, puis on les classe par ordre décroissant.

```
SELECT name, salary
FROM teacher t1
WHERE 2 >= (
    SELECT COUNT(DISTINCT t2.salary)
    FROM teacher t2
    WHERE t2.salary < t1.salary
)
ORDER BY salary DESC;
```

FIGURE 15 – Script SQL de la question 8

NAME	SALARY
Califieri	62000
El Said	60000
Mozart	40000

FIGURE 16 – Résultat de la question 8

Q9. Afficher les noms des étudiants qui ont suivi un cours en automne 2009 avec IN

Cette requête teste si le couple semestre–année appartient aux inscriptions de chaque étudiant.

```
SELECT s.name
FROM student s
WHERE ('Fall', 2009) IN (
    SELECT t.semester, t.year
    FROM takes t
    WHERE t.id = s.id
);
```

FIGURE 17 – Script SQL de la question 9

NAME
Zhang
Shankar
Peltier
Levy
Williams
Brown
Bourikas

7 rows returned in 0.08 seconds [Download](#)

FIGURE 18 – Résultat de la question 9

Q10. Afficher les noms des étudiants qui ont suivi un cours en automne 2009 avec SOME

Cette question reprend la précédente avec une autre écriture logique basée sur l'opérateur SOME.

```
SELECT s.name
FROM student s
WHERE ('Fall', 2009) = SOME (
    SELECT t.semester, t.year
    FROM takes t
    WHERE t.id = s.id
);
```

FIGURE 19 – Script SQL de la question 10

NAME
Zhang
Shankar
Peltier
Levy
Williams
Brown
Bourikas

FIGURE 20 – Résultat de la question 10

Q11. Afficher les noms des étudiants qui ont suivi un cours en automne 2009 avec NATURAL INNER JOIN

La jointure naturelle relie ici les tables ayant l'attribut id en commun.

```
SELECT DISTINCT name
FROM student
NATURAL INNER JOIN takes
WHERE semester = 'Fall'
AND year = 2009;
```

FIGURE 21 – Script SQL de la question 11

NAME
Zhang
Shankar
Peltier
Levy
Williams
Brown
Bourikas
7 rows returned in 0.03 seconds Download

FIGURE 22 – Résultat de la question 11

Q12. Afficher les noms des étudiants qui ont suivi un cours en automne 2009 avec EXISTS

La clause EXISTS permet ici de vérifier qu'une inscription en automne 2009 existe pour l'étudiant considéré.

```
SELECT s.name
FROM student s
WHERE EXISTS (
    SELECT 1
    FROM takes t
    WHERE t.id = s.id
        AND t.semester = 'Fall'
        AND t.year = 2009
);
```

FIGURE 23 – Script SQL de la question 12

NAME
Zhang
Shankar
Peltier
Levy
Williams
Brown
Bourikas

FIGURE 24 – Résultat de la question 12

Q13. Afficher toutes les paires d'étudiants ayant suivi au moins un cours ensemble

Cette auto-jointure permet de former les couples d'étudiants ayant partagé au moins une même section de cours.

```

SELECT s1.name AS student_1,
       s2.name AS student_2
FROM takes t1
JOIN takes t2
  ON t1.course_id = t2.course_id
 AND t1.sec_id = t2.sec_id
 AND t1.semester = t2.semester
 AND t1.year = t2.year
 AND t1.id < t2.id
JOIN student s1
  ON s1.id = t1.id
JOIN student s2
  ON s2.id = t2.id
GROUP BY s1.name, s2.name
HAVING COUNT(*) >= 1
ORDER BY s1.name, s2.name;

```

FIGURE 25 – Script SQL de la question 13

STUDENT_1	STUDENT_2
Levy	Bourikas
Levy	Brown
Levy	Williams
Shankar	Bourikas
Shankar	Brown
Shankar	Levy
Shankar	Williams
Williams	Bourikas
Williams	Brown

FIGURE 26 – Résultat de la question 13

Q14. Afficher pour chaque enseignant ayant assuré un cours le nombre total d'étudiants ayant suivi ses cours

Cette requête compte le nombre total d'inscriptions associées à chaque enseignant.

```
SELECT t.name,  
       COUNT(*) AS total_students  
FROM teacher t  
JOIN teaches te  
  ON t.id = te.id  
JOIN takes ta  
  ON te.course_id = ta.course_id  
  AND te.sec_id = ta.sec_id  
  AND te.semester = ta.semester  
  AND te.year = ta.year  
GROUP BY t.id, t.name  
ORDER BY total_students DESC;
```

FIGURE 27 – Script SQL de la question 14

NAME	TOTAL_STUDENTS
Srinivasan	10
Brandt	3
Crick	2
Katz	2
Kim	1
Einstein	1
Mozart	1
Wu	1
El Said	1

2 rows returned in 0.66 seconds [Download](#)

FIGURE 28 – Résultat de la question 14

Q15. Afficher pour chaque enseignant, même s'il n'a pas assuré de cours, le nombre total d'étudiants ayant suivi ses cours

L'utilisation de jointures externes permet de conserver également les enseignants sans cours.

```
SELECT t.name,  
       COUNT(ta.course_id) AS total_students  
FROM teacher t  
LEFT JOIN teaches te  
  ON t.id = te.id  
LEFT JOIN takes ta  
  ON te.course_id = ta.course_id  
  AND te.sec_id = ta.sec_id  
  AND te.semester = ta.semester  
  AND te.year = ta.year  
GROUP BY t.id, t.name  
ORDER BY total_students DESC;
```

FIGURE 29 – Script SQL de la question 15

NAME	TOTAL_STUDENTS
Srinivasan	10
Brandt	3
Katz	2
Crick	2
Einstein	1
Mozart	1
Klm	1
Wu	1
El Said	1

FIGURE 30 – Résultat de la question 15

Q16. Pour chaque enseignant, afficher le nombre total de grades A attribués

Cette requête compte le nombre de notes égales à A attribuées par chaque enseignant.

```
SELECT t.name,  
       COUNT(*) AS total_a  
FROM teacher t  
JOIN teaches te  
  ON t.id = te.id  
JOIN takes ta  
  ON te.course_id = ta.course_id  
  AND te.sec_id = ta.sec_id  
  AND te.semester = ta.semester  
  AND te.year = ta.year  
WHERE ta.grade = 'A'  
GROUP BY t.id, t.name  
ORDER BY total_a DESC;
```

FIGURE 31 – Script SQL de la question 16

NAME	TOTAL_A
Srinivasan	4
Brandt	2
Crick	1

3 rows returned in 0.00 seconds [Download](#)

FIGURE 32 – Résultat de la question 16

Q17. Afficher toutes les paires enseignant–élève ainsi que le nombre de fois où l’élève a suivi un cours dispensé par cet enseignant

Le résultat associe chaque enseignant à chaque étudiant concerné, avec le nombre de cours suivis ensemble.

```

SELECT t.name AS teacher_name,
       s.name AS student_name,
       COUNT(*) AS nb_times
FROM teacher t
JOIN teaches te
  ON t.id = te.id
JOIN takes ta
  ON te.course_id = ta.course_id
  AND te.sec_id = ta.sec_id
  AND te.semester = ta.semester
  AND te.year = ta.year
JOIN student s
  ON s.id = ta.id
GROUP BY t.name, s.name
ORDER BY t.name, s.name;

```

FIGURE 33 – Script SQL de la question 17

TEACHER_NAME	STUDENT_NAME	NB_TIMES
Brandt	Williams	1
Crick	Tanaka	2
Einstein	Peltier	1
El Said	Brandt	1
Katz	Levy	2
Kim	Aoi	1
Mozart	Sanchez	1
Srinivasan	Bourikas	2

More than 10 rows available. Increase rows selector to view more rows.

1 rows returned in 0.14 seconds. [Download](#)

FIGURE 34 – Résultat de la question 17

Q18. Afficher toutes les paires enseignant–élève où un élève a suivi au moins deux cours dispensés par cet enseignant

Cette dernière requête reprend la précédente en ne conservant que les couples apparaissant au moins deux fois.

```
SELECT teacher_name, student_name
FROM (
    SELECT t.name AS teacher_name,
           s.name AS student_name,
           COUNT(*) AS nb_times
    FROM teacher t
    JOIN teaches te
      ON t.id = te.id
    JOIN takes ta
      ON te.course_id = ta.course_id
      AND te.sec_id = ta.sec_id
      AND te.semester = ta.semester
      AND te.year = ta.year
    JOIN student s
      ON s.id = ta.id
    GROUP BY t.name, s.name
)
WHERE nb_times >= 2
ORDER BY teacher_name, student_name;
```

FIGURE 35 – Script SQL de la question 18

TEACHER_NAME	STUDENT_NAME
Crick	Tanaka
Katz	Levy
Srinivasan	Bourikas
Srinivasan	Shankar
Srinivasan	Zhang

rows returned in 0.02 seconds [Download](#)

FIGURE 36 – Résultat de la question 18

2 Exercice 2 : Dépendances fonctionnelles et normalisation

1. $R(A, B, C)$ avec $F = \{A \rightarrow B, B \rightarrow C\}$

On calcule :

$$A^+ = \{A, B, C\}$$

Donc A est une clé candidate.

La relation est en deuxième forme normale, mais elle n'est pas en troisième forme normale à cause de la dépendance transitive :

$$A \rightarrow B \quad \text{et} \quad B \rightarrow C$$

La forme normale la plus avancée est donc la **2FN**.

Une décomposition correcte est :

$$R_1(A, B), \quad R_2(B, C)$$

2. $R(A, B, C)$ avec $F = \{A \rightarrow C, A \rightarrow B\}$

On a :

$$A^+ = \{A, B, C\}$$

Donc A est une clé candidate.

Toutes les dépendances fonctionnelles ont une clé candidate à gauche. La relation est donc en **BCNF**.

3. $R(A, B, C)$ avec $F = \{AB \rightarrow C, C \rightarrow B\}$

On a :

$$(AB)^+ = \{A, B, C\}$$

donc AB est une clé candidate.

De plus :

$$(AC)^+ = \{A, C\}$$

et comme $C \rightarrow B$, on obtient :

$$(AC)^+ = \{A, B, C\}$$

Donc AC est aussi une clé candidate.

La dépendance

$$C \rightarrow B$$

viole la BCNF car C n'est pas une super-clé.

La relation reste cependant en **3FN**, puisque B est un attribut premier.

Une décomposition en BCNF est :

$$R_1(C, B), \quad R_2(A, C)$$

3 Exercice 3 : Fermetures, BCNF et décomposition

1. Dédire au moins 16 dépendances fonctionnelles

On considère :

$$F = \{A \rightarrow B, C; CD \rightarrow E; B \rightarrow D; E \rightarrow A\}$$

À partir de ces dépendances, on peut déduire par exemple :

$$\begin{aligned} &A \rightarrow B, \quad A \rightarrow C, \quad B \rightarrow D, \quad CD \rightarrow E, \quad E \rightarrow A, \\ &A \rightarrow D, \quad A \rightarrow BC, \quad A \rightarrow CD, \quad A \rightarrow E, \quad E \rightarrow B, \\ &E \rightarrow C, \quad E \rightarrow D, \quad A \rightarrow A, \quad B \rightarrow B, \quad C \rightarrow C, \quad D \rightarrow D, \quad E \rightarrow E \end{aligned}$$

2. Soit $R(A, B, C, D, E, F)$ avec $F = \{A \rightarrow B, C, D; BC \rightarrow D, E; B \rightarrow D; D \rightarrow A\}$

a) Calcul de B^+ et $(AB)^+$

On obtient :

$$B^+ = \{A, B, C, D, E\}$$

et :

$$(AB)^+ = \{A, B, C, D, E\}$$

b) Montrer que $\{A, F\}$ est une super-clé

On calcule :

$$(AF)^+ = \{A, B, C, D, E, F\}$$

Donc $\{A, F\}$ est une super-clé.

c) La relation est-elle en BCNF ?

La relation n'est pas en BCNF, car certaines dépendances ont un membre gauche qui n'est pas une super-clé, comme par exemple :

$$A \rightarrow B, C, D$$

Une décomposition possible en BCNF est :

$$(B, D), \quad (A, B, C), \quad (E, A), \quad (E, F)$$

3. Décomposition sans perte et avec perte

a) Décomposition $R_1(A, B, C)$ et $R_2(A, D, E)$

L'intersection des deux relations est :

$$R_1 \cap R_2 = \{A\}$$

Cette décomposition est **sans perte d'information**.

b) Décomposition $R_1(A, B, C)$ et $R_2(C, D, E)$

Ici :

$$R_1 \cap R_2 = \{C\}$$

Cette décomposition est **avec perte d'information**.

4 Exercice 4 : Implémentation en Python

Dans cet exercice, les différentes fonctions ont été implémentées en Python dans VS Code, puis testées dans le terminal. Pour chaque question, une première capture présente le code correspondant et une seconde capture montre le résultat obtenu à l'exécution.

Q1. Afficher une liste de dépendances fonctionnelles

Cette première question consiste à écrire une fonction permettant d'afficher correctement un ensemble de dépendances fonctionnelles.

```
def printDependencies(F):  
    for alpha, beta in F:  
        print("\t", format_set(alpha), "-->", format_set(beta))
```

FIGURE 37 – Code Python de la question 1

```
1) Affichage des dépendances fonctionnelles :  
    {A} --> {B}  
    {A} --> {C}  
    {C, G} --> {H}  
    {C, G} --> {I}  
    {B} --> {H}
```

FIGURE 38 – Résultat de la question 1

Q2. Afficher un ensemble de relations

Cette question demande d'écrire une fonction qui affiche un ensemble de relations sous une forme lisible.

```
def printRelations(T):  
    for R in T:  
        print("\t", format_set(R))
```

FIGURE 39 – Code Python de la question 2

```
2) Affichage des relations :  
    {A, B, C, G, H, I}  
    {X, Y}
```

FIGURE 40 – Résultat de la question 2

Q3. Écrire une fonction qui retourne tous les sous-ensembles d'un ensemble

Le but ici est de générer l'ensemble des parties d'un ensemble donné.

```
def powerSet(inputset: set):  
    result = []  
    elements = list(inputset)  
  
    for r in range(1, len(elements) + 1):  
        for comb in itertools.combinations(elements, r):  
            result.append(set(comb))  
  
    return result
```

FIGURE 41 – Code Python de la question 3

```
3) Ensemble des parties de {A, B, C} :  
Sous-ensembles :  
    {A}  
    {C}  
    {B}  
    {A, C}  
    {A, B}  
    {B, C}  
    {A, B, C}
```

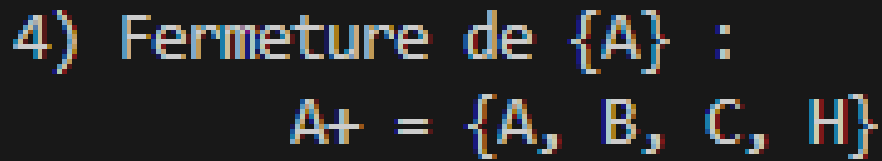
FIGURE 42 – Résultat de la question 3

Q4. Calculer la fermeture d'un ensemble d'attributs

Cette fonction calcule la fermeture d'un ensemble d'attributs à partir d'un ensemble de dépendances fonctionnelles.

```
def computeAttributeClosure(F, K: set):  
    K_plus = set(K)  
    size = -1  
  
    while size != len(K_plus):  
        size = len(K_plus)  
        for alpha, beta in F:  
            if alpha.issubset(K_plus):  
                K_plus.update(beta)  
  
    return K_plus
```

FIGURE 43 – Code Python de la question 4

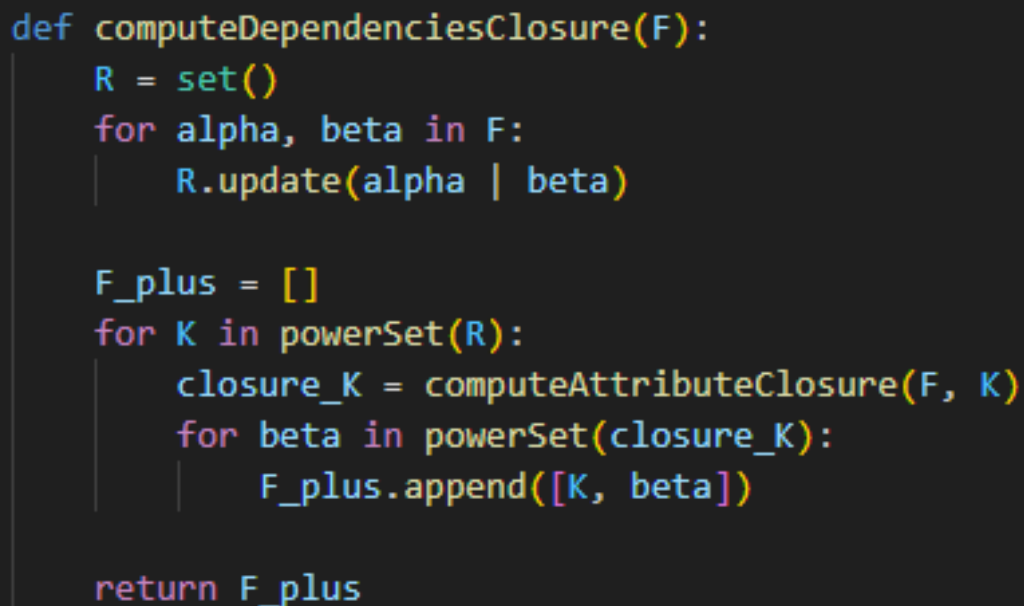


4) Fermeture de $\{A\}$:
 $A^+ = \{A, B, C, H\}$

FIGURE 44 – Résultat de la question 4

Q5. Calculer la clôture de F

Cette question consiste à produire l'ensemble des dépendances fonctionnelles déductibles à partir de F .



```
def computeDependenciesClosure(F):  
    R = set()  
    for alpha, beta in F:  
        R.update(alpha | beta)  
  
    F_plus = []  
    for K in powerSet(R):  
        closure_K = computeAttributeClosure(F, K)  
        for beta in powerSet(closure_K):  
            F_plus.append([K, beta])  
  
    return F_plus
```

FIGURE 45 – Code Python de la question 5

5) Clôture de F (affichage partiel) :
Quelques dépendances de F^+ :

1. $\{I\} \twoheadrightarrow \{I\}$
2. $\{A\} \twoheadrightarrow \{A\}$
3. $\{A\} \twoheadrightarrow \{C\}$
4. $\{A\} \twoheadrightarrow \{B\}$
5. $\{A\} \twoheadrightarrow \{H\}$
6. $\{A\} \twoheadrightarrow \{A, C\}$
7. $\{A\} \twoheadrightarrow \{A, B\}$
8. $\{A\} \twoheadrightarrow \{A, H\}$
9. $\{A\} \twoheadrightarrow \{B, C\}$
10. $\{A\} \twoheadrightarrow \{C, H\}$
11. $\{A\} \twoheadrightarrow \{B, H\}$
12. $\{A\} \twoheadrightarrow \{A, B, C\}$
13. $\{A\} \twoheadrightarrow \{A, C, H\}$
14. $\{A\} \twoheadrightarrow \{A, B, H\}$
15. $\{A\} \twoheadrightarrow \{B, C, H\}$

FIGURE 46 – Résultat de la question 5

Q6. Tester si $\alpha \rightarrow \beta$

Cette fonction permet de vérifier si une dépendance fonctionnelle donnée est vérifiée.

```
def isDependency(F, alpha: set, beta: set):
    return beta.issubset(computeAttributeClosure(F, alpha))
```

FIGURE 47 – Code Python de la question 6

6) Test de dépendance fonctionnelle $\{A\} \rightarrow \{H\}$:
True

FIGURE 48 – Résultat de la question 6

Q7. Tester si K est une super-clé

Cette fonction vérifie si un ensemble d'attributs détermine tous les attributs d'une relation.


```
def isSuperKey(F, R: set, K: set):  
    return R.issubset(computeAttributeClosure(F, K))
```

FIGURE 49 – Code Python de la question 7

```
7) Test super-clé pour R = {A,B,C,G,H,I} et K = {A,G} :  
    True
```

FIGURE 50 – Résultat de la question 7

Q8. Tester si K est une clé candidate

Cette question consiste à vérifier si une super-clé est minimale, c'est-à-dire si elle constitue une clé candidate.

```
def isCandidateKey(F, R: set, K: set):  
    if not isSuperKey(F, R, K):  
        return False  
  
    for A in K:  
        K1 = set(K)  
        K1.discard(A)  
        if isSuperKey(F, R, K1):  
            return False  
  
    return True
```

FIGURE 51 – Code Python de la question 8

```
8) Test clé candidate pour R = {A,B,C,G,H,I} et K = {A,G} :  
    True
```

FIGURE 52 – Résultat de la question 8

Q9. Calculer toutes les clés candidates

Cette fonction parcourt les sous-ensembles d'attributs et conserve ceux qui sont des clés candidates.

```
def computeAllCandidateKeys(F, R: set):  
    result = []  
    for K in powerSet(R):  
        if isCandidateKey(F, R, K):  
            result.append(K)  
    return result
```

FIGURE 53 – Code Python de la question 9

```
9) Toutes les clés candidates de R = {A,B,C,G,H,I} :  
Clés candidates :  
    {A, G}
```

FIGURE 54 – Résultat de la question 9

Q10. Calculer toutes les super-clés

Cette question demande de déterminer tous les ensembles d'attributs qui déterminent toute la relation.

```
def computeAllSuperKeys(F, R: set):  
    result = []  
    for K in powerSet(R):  
        if isSuperKey(F, R, K):  
            result.append(K)  
    return result
```

FIGURE 55 – Code Python de la question 10

```
10) Toutes les super-clés de R = {A,B,C,G,H,I}
Super-clés :
    {A, G}
    {A, G, I}
    {A, C, G}
    {A, B, G}
    {A, G, H}
    {A, C, G, I}
    {A, B, G, I}
    {A, G, H, I}
    {A, B, C, G}
    {A, C, G, H}
    {A, B, G, H}
    {A, B, C, G, I}
    {A, C, G, H, I}
    {A, B, G, H, I}
    {A, B, C, G, H}
    {A, B, C, G, H, I}
```

FIGURE 56 – Résultat de la question 10

Q11. Retourner une clé candidate

L'objectif de cette fonction est de renvoyer au moins une clé candidate d'une relation.

```
def computeOneCandidateKey(F, R: set):
    K = set(R)

    while not isCandidateKey(F, R, K):
        for A in list(K):
            if isSuperKey(F, R, K.difference({A})):
                K.remove(A)
                break

    return K
```

FIGURE 57 – Code Python de la question 11

11) Une clé candidate de $R = \{A, B, C, G, H, I\}$:
 $\{A, G\}$

FIGURE 58 – Résultat de la question 11

Q12. Tester si une relation est en BCNF

Cette fonction vérifie si une relation respecte la forme normale de Boyce-Codd.

```
def isBCNFRelation(F, R: set):  
    for K in powerSet(R):  
        K_plus = computeAttributeClosure(F, K)  
        Y = K_plus.difference(K)  
  
        if (not R.issubset(K_plus)) and (not Y.isdisjoint(R)):  
            return False, [K, Y & R]  
  
    return True, [set(), set()]
```

FIGURE 59 – Code Python de la question 12

12) BCNF pour $R = \{A, B, C, G, H, I\}$:
Est en BCNF ? False

FIGURE 60 – Résultat de la question 12

Q13. Tester si un schéma de relations est en BCNF

Cette question généralise le test précédent à un ensemble de relations.

```
def isBCNFRelations(F, T):  
    for R in T:  
        ok, _ = isBCNFRelation(F, R)  
        if not ok:  
            return False, R  
  
    return True, set()
```

FIGURE 61 – Code Python de la question 13

```
13) BCNF pour le schéma myrelations :  
Schéma en BCNF ? False  
Relation problématique : {'I', 'A', 'C', 'B', 'H', 'G'}
```

FIGURE 62 – Résultat de la question 13

Q14. Décomposition en BCNF

Cette dernière fonction applique une décomposition progressive jusqu'à obtenir un schéma de relations respectant la BCNF.

```
def computeBCNFDecomposition(F, T):  
    OUT = list(T)  
    size = -1  
  
    while size != len(OUT):  
        size = len(OUT)  
  
        for R in OUT:  
            ok, data = isBCNFRelation(F, R)  
            alpha, beta = data  
  
            if not ok:  
                r1 = alpha | beta  
                r2 = R.difference(beta)  
  
                if r1 not in OUT:  
                    OUT.append(r1)  
                if r2 not in OUT:  
                    OUT.append(r2)  
  
                OUT.remove(R)  
                break  
  
    return OUT
```

FIGURE 63 – Code Python de la question 14

```
14) Décomposition en BCNF du schéma myrelations :  
    {X, Y}  
    {A, G, I}  
    {B, H}  
    {A, B, C}
```

FIGURE 64 – Résultat de la question 14